

Domain vs Service

Where does my trigger logic go, and what do I put in a domain class vs a service class?

FFLib Series · Session 003

John M. Daniel

Code With Sally



About me

**Senior Director of Digital Platforms, Steampunk, Inc. |
Apex Enterprise Patterns Maintainer & Architect**

- **Apex Enterprise Patterns (fflib) maintainer & architect** — steward of `fflib-apex-common`, `fflib-apex-mocks`, `force-di`, and `AT4DX` at github.com/apex-enterprise-patterns
- **Open source education** — promoting awareness and adoption of Domain, Service, and Selector patterns for enterprise Salesforce development
- **Senior Director, Digital Platforms** — leading Salesforce platform engineering standards and delivery at Steampunk, Inc.

When I am not working on Apex architecture, I am probably driving my Jeep on the beach.



FFLib Series

SESSION



Session #1 — Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 — Service Layers Explained: Coordinating Business Logic in Apex



Session #3 — Domain vs Service: Where Should Your Apex Logic Live?



Session #4 — Query Logic as a First-Class Architecture Concern



Session #5 — Mocking in Apex: Why It Changes Everything

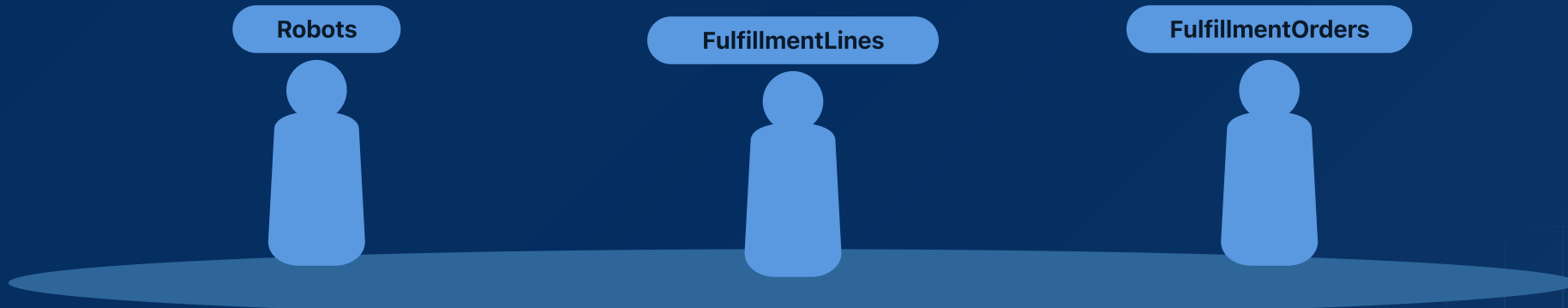


Session #6 — Enterprise-Scale Apex Across Multiple Packages

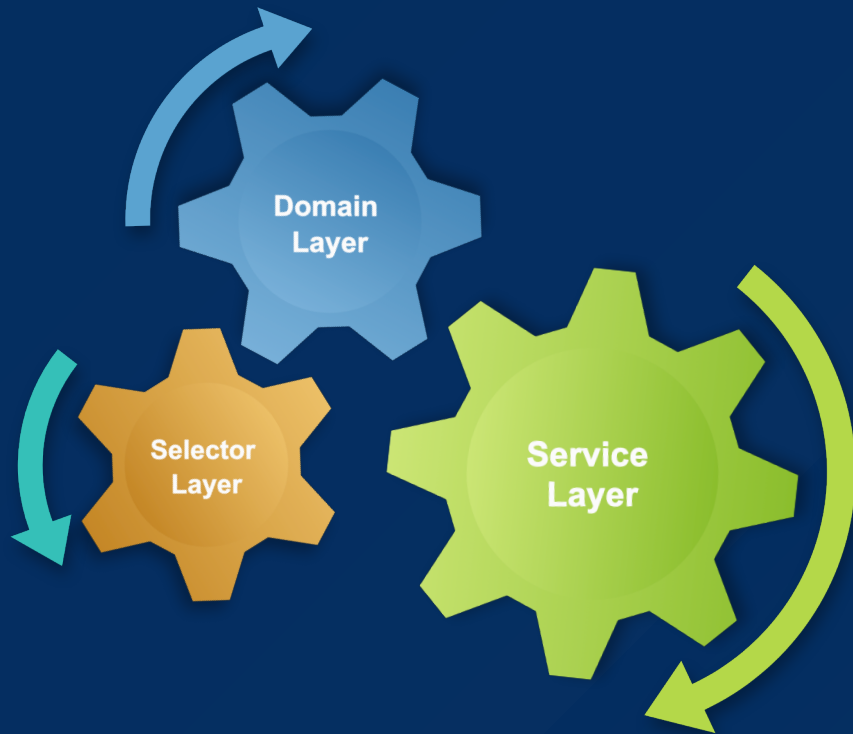
Domain Layer

“ *A single instance that handles the business logic for all rows in a database table or view.* ”

[Martin Fowler – Table Module](#) – *Patterns of Enterprise Application Architecture*



Domain vs Service



DOMAIN VS SERVICE

Recap - Service Layer



Domain Principles



Warehouse App Objects and Behaviors



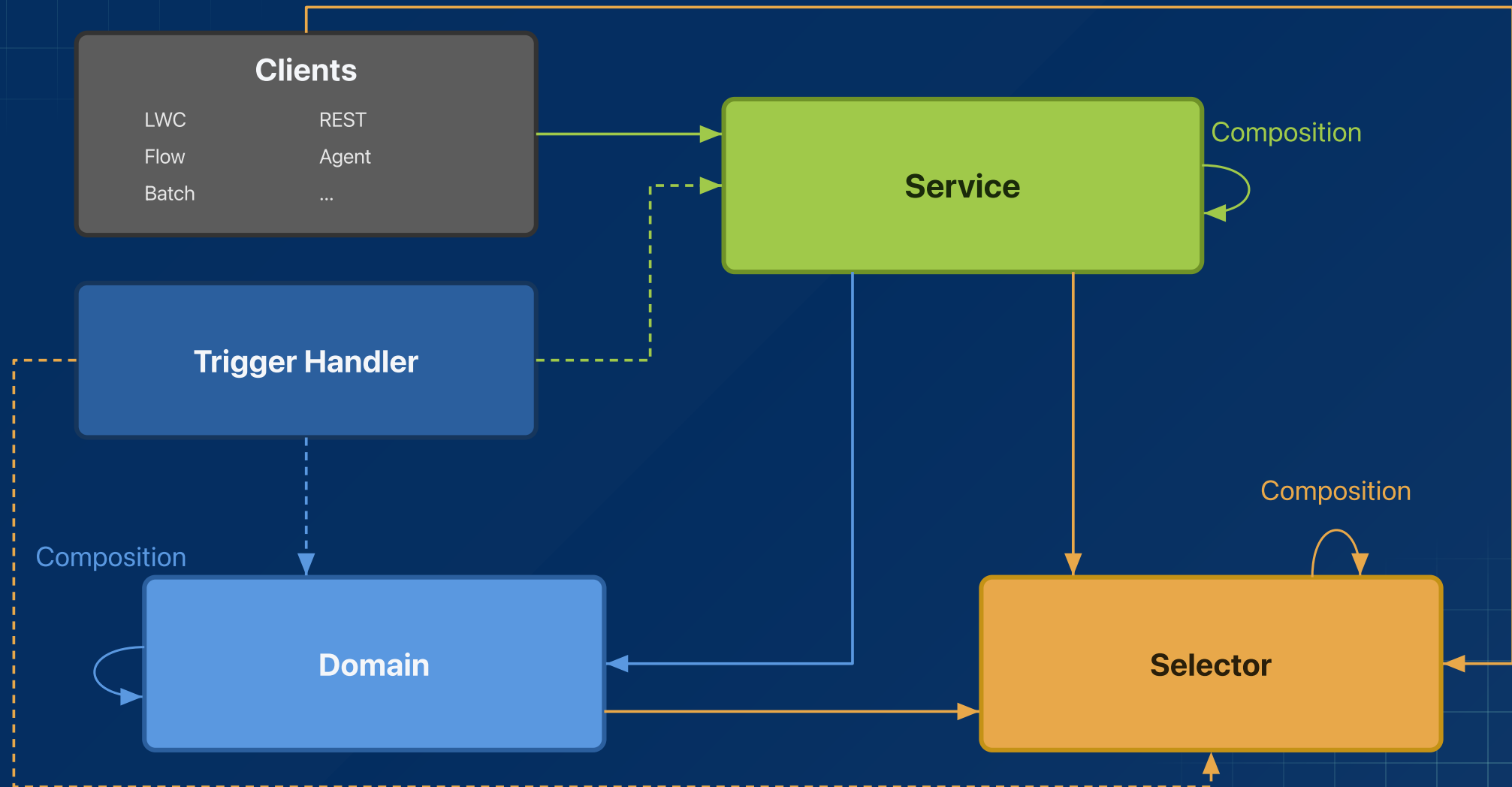
Warehouse App Domain vs Trigger Code



Recap - Service Layer

- **Task-oriented** — a Service represents a feature; its methods expose business tasks.
 - `DispatchService.dispatchWarehouses(...)`
- **Reusable entry point** — LWC, REST, Flow, Agent actions, and Batch call the same Service.
- **Coordinates the work** — Selectors query; Domains apply object-specific behavior.
- **Owns the transaction** — create a Unit of Work, register changes, then commit once.

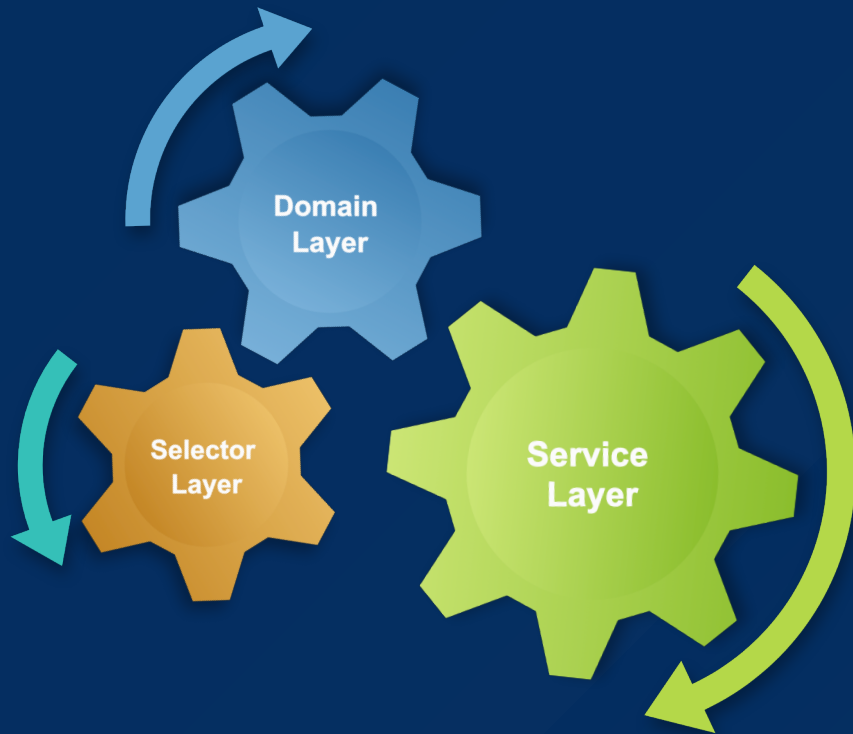
Who can call whom



Who can call whom

CALLER	Service	Domain	Selector
Client (LWC, REST, Flow, Agent, Batch, ...)	•		•
Trigger Handler	•	•	•
Service	•	•	•
Domain		•	•
Selector			•

Domain vs Service



DOMAIN VS SERVICE

Recap - Service Layer



Domain Principles



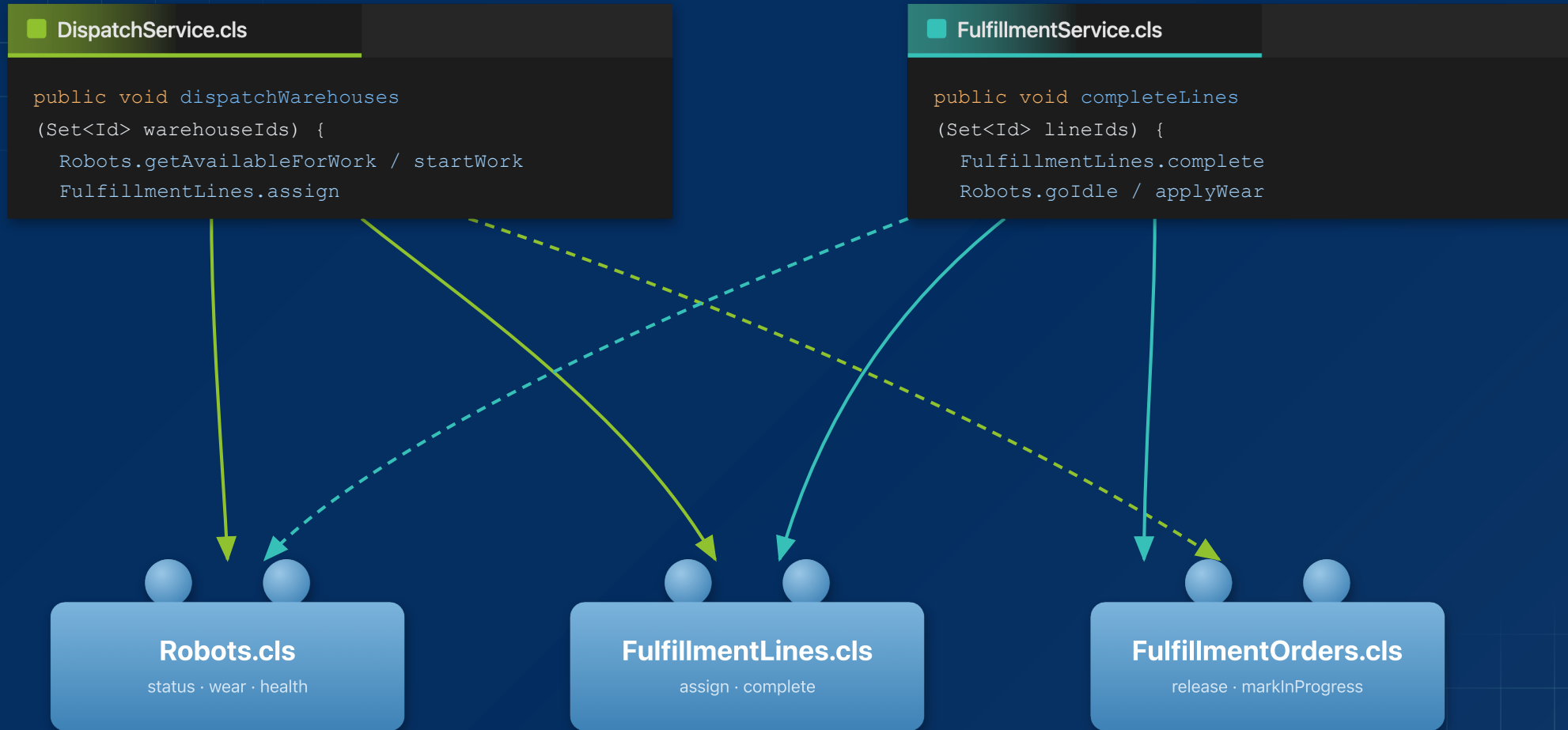
Warehouse App Objects and Behaviors



Warehouse App Domain vs Trigger Code



Domain classes are like Lego bricks



Same three bricks. Two different builds.

No one re-implemented robot wear, line status, or order status for the second function.

Domain ➔ Object Orientated Logic

- A Domain class combines **data and behavior** of an object — one Lego brick
 - Robots — `Robots.newInstance(List<Robot__c> robots)`
 - Fulfillment Orders —
`FulfillmentOrders.newInstance(List<FulfillmentOrder__c> orders)`
 - Base domain classes extend `fflib_SObjects` — no trigger plumbing required
- **Methods** are ways to interact with the object's behaviors —
`robots.startWork(uow)` , `fulfillmentLines.complete(uow)`
- Trigger / CRUD responses are a **separate concern** in this sample
 - A **sidecar handler** class extends `fflib_SObjectDomain` — `RobotsTriggerHandler`
 - Wired from a thin trigger —
`fflib_SObjectDomain.triggerHandler(RobotsTriggerHandler.class)`

Domain Checklist

- Record collections — wrap many records, not one
 -  `Robots.newInstance(List<Robot__c> robots)`
 -  `Robots.newInstance(Robot__c robot)`
- In-memory behavior, registered not committed — no direct DML
 -  `void startWork(fflib_SObjectUnitOfWork uow)`
 -  `void startWork() { update robots; }`
- **Records encapsulated** — don't have callers pass them in
 -  `robots.startWork(uow)`
 -  `startWork(Robot__c robot, uow)`
- **Trigger logic** — optional sidecar handler, e.g. `RobotsTriggerHandler`
- **Object-oriented** — data and behavior stay together — the reusable brick
- **Security** — SOQL and DML are user mode, unless Apex Trigger context

Domain Evolved

- **Domains can be just behavior** – operation methods only, no CRUD plumbing required
 -  `Robots.cls` extends `fplib_SObjects` – `startWork`, `goIdle`, `applyWear`
 -  One class handling both business rules and every trigger event
- **Sidecar trigger handlers** – fine for object logic tied to CRUD
 -  `RobotsTriggerHandler.cls` extends `fplib_SObjectDomain` – `onApplyDefaults`, `onValidate`, `onBeforeUpdate`
 -  Defaulting and validation logic duplicated inside the Domain and the trigger
- **Combined domains** – still fine if a class does both; this sample chooses the split for clarity

① Application class

Optional – this sample uses concrete `newInstance()` factories and a `@TestVisible` mock, no `Application` factory required.

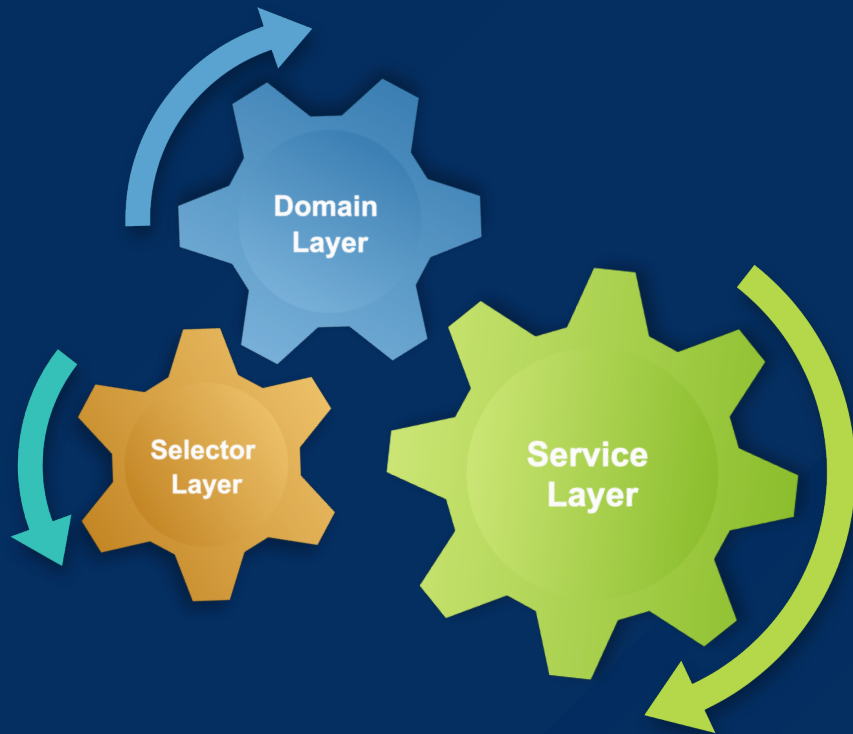
```
1 public virtual inherited sharing class Robots extends fflib_SObjects {  
2  
3     ...  
4  
5     @TestVisible  
6     private static Robots mock;  
7  
8     public static Robots newInstance(List<Robot__c> recordList) {  
9         return mock != null ? mock : new Robots(recordList);  
10    }  
11  
12    public Robots(List<Robot__c> sObjectList) {  
13        super(sObjectList == null ? new List<Robot__c>() : sObjectList);  
14    }
```

```
1 public virtual void startWork(fflib_SObjectUnitOfWork uow) {
2     for (Robot__c robot : (List<Robot__c>) getRecords()) {
3         // criteria
4         if (!canAcceptWork(robot)) {
5             throw new RobotsException(
6                 'Robot ' + robot.Name + ' cannot accept work.'
7             );
8         }
9         // action
10        robot.Status__c = STATUS_WORKING;
11        registerStatusDirty(uow, robot);
12    }
13 }
```

■ DispatchService.dispatchWarehouses

```
1 public virtual void dispatchWarehouses(Set<Id> warehouseIds) {
2     fflib_SObjectUnitOfWork uow = UnitOfWork.newInstance();
3     ...
4     List<FulfillmentLine__c> pending =
5         lineSelector.selectPendingUnassignedByWarehouseWithOrder(warehouseIds);
6
7     List<Robot__c> idle = robotSelector.selectIdleByWarehouseWithModel(warehouseIds);
8
9     // Robot domain: keep only robots that can legally accept work
10    List<Robot__c> available = Robots.newInstance(idle).getAvailableForWork();
11
12    Map<Id, Id> robotIdByLineId = match(pending, available);
13    ...
14    Set<Id> assignedRobotIds = new Set<Id>(robotIdByLineId.values());
15
16    List<Robot__c> robotsToStart = new List<Robot__c>();
17    for (Robot__c robot : available) {
18        if (assignedRobotIds.contains(robot.Id)) {
19            robotsToStart.add(robot);
20        }
21    }
22
23    Robots.newInstance(robotsToStart).startWork(uow);
24    ...
25    FulfillmentLines.newInstance(linesToAssign).assign(robotIdByLineId, uow);
```

Domain vs Service



DOMAIN VS SERVICE

Recap - Service Layer



Domain Principles



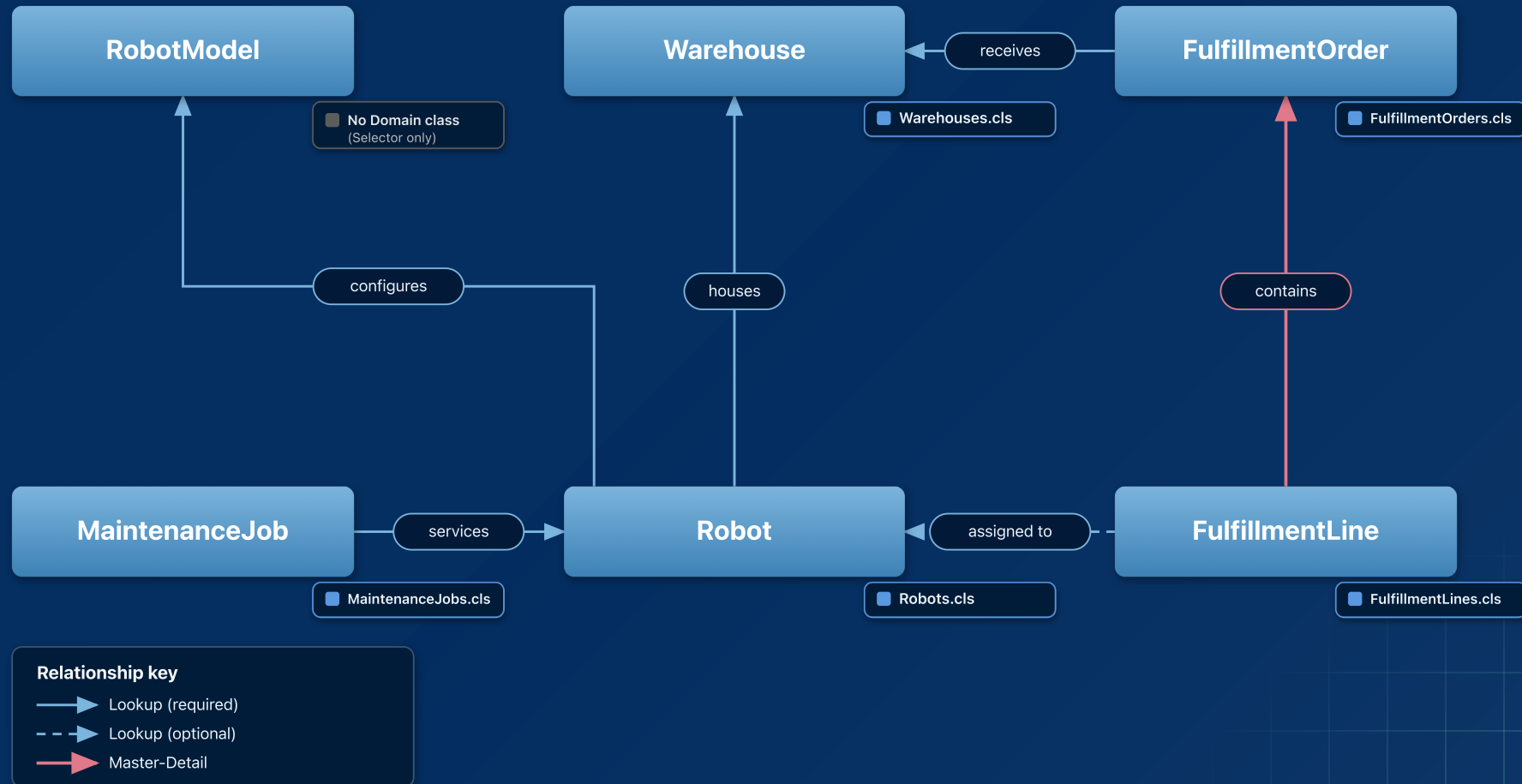
Warehouse App Objects and Behaviors



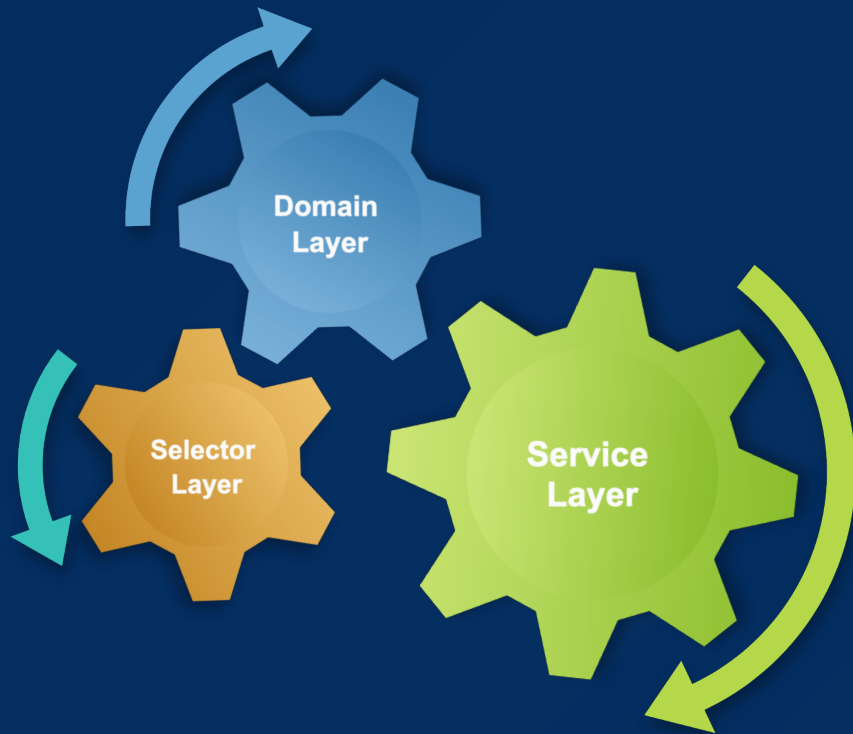
Warehouse App Domain vs Trigger Code



Warehouse App – Domain Objects



Domain vs Service



DOMAIN VS SERVICE

Recap - Service Layer



Domain Principles



Warehouse App Objects and Behaviors



Warehouse App Domain vs Trigger Code



Trigger Handlers vs Domain

Robots.cls (Domain) owns object behavior, callable any time. RobotsTriggerHandler.cls (sidecar) owns the trigger-context responses — defaults, validation — wired in by a one-line trigger.

Service Layer → Domain

■ Robots.cls

```
1 public virtual inherited sharing class Robots
2     extends fflib_SObjects {
3
4     public virtual void startWork(
5         fflib_SObjectUnitOfWork uow)
6     public virtual void goIdle(
7         fflib_SObjectUnitOfWork uow)
8     public virtual void applyWear(
9         Map<Id, Decimal> hoursByRobotId,
10        fflib_SObjectUnitOfWork uow)
11     public virtual List<Robot__c> getAvailableForWork()
12 }
```

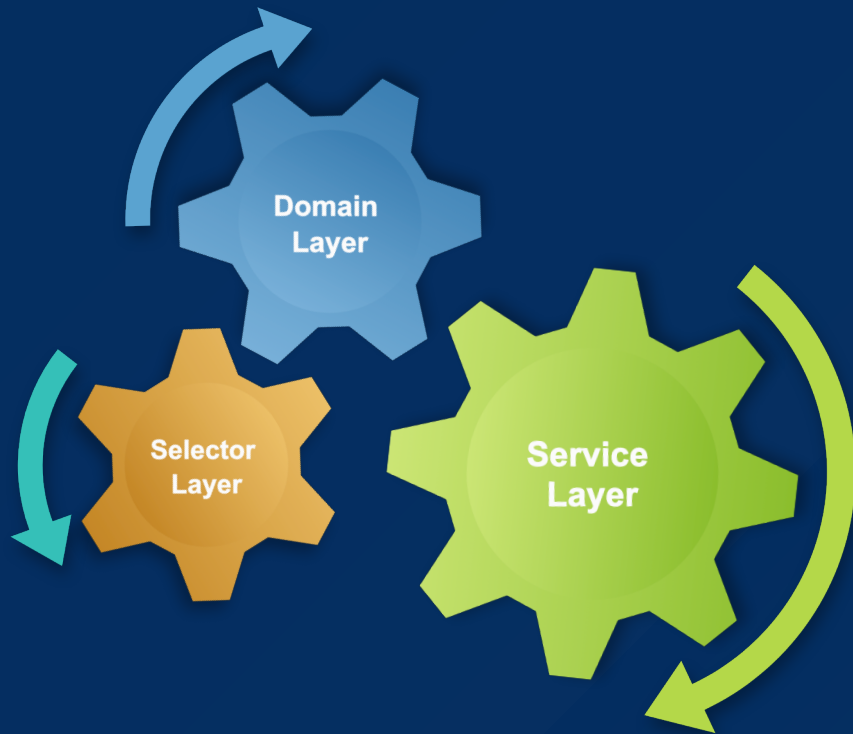
Apex Trigger → Trigger Handler → Domain

■ RobotsTriggerHandler.cls

```
1 public inherited sharing class RobotsTriggerHandler
2     extends fflib_SObjectDomain {
3
4     public override void onApplyDefaults()
5     public override void onValidate(
6         Map<Id, SObject> existingRecords)
7     public override void onBeforeUpdate(
8         Map<Id, SObject> existingRecords)
9 }
```

Robots.trigger is one line: `fflib_SObjectDomain.triggerHandler(RobotsTriggerHandler.class)`. The handler only defaults, validates, and reacts to the trigger event.

Domain vs Service



DOMAIN VS SERVICE

Recap - Service Layer



Domain Principles



Warehouse App Objects and Behaviors



Warehouse App Domain vs Trigger Code



What's Next — FFLib Series

SESSION



Session #1 — Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 — Service Layers Explained: Coordinating Business Logic in Apex



Session #3 — Domain vs Service: Where Should Your Apex Logic Live?



Session #4 — Query Logic as a First-Class Architecture Concern



Session #5 — Mocking in Apex: Why It Changes Everything



Session #6 — Enterprise-Scale Apex Across Multiple Packages

Thank you!

John M. Daniel • Code With Sally

FFLib Series • Session 003

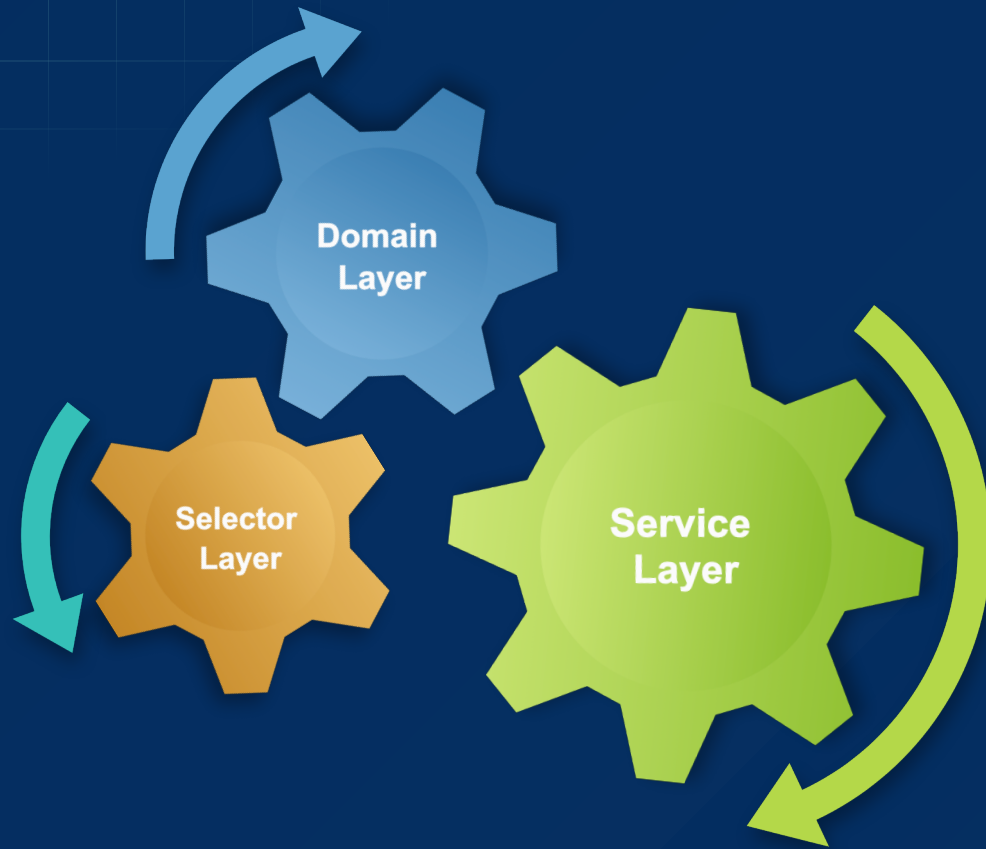


Where should your Apex logic live?

“ *One of the biggest challenges in Apex architecture is deciding where logic belongs. This session explores the responsibilities of domain classes, service classes, and triggers so you can build systems that are easier to understand, extend, and test.* ”

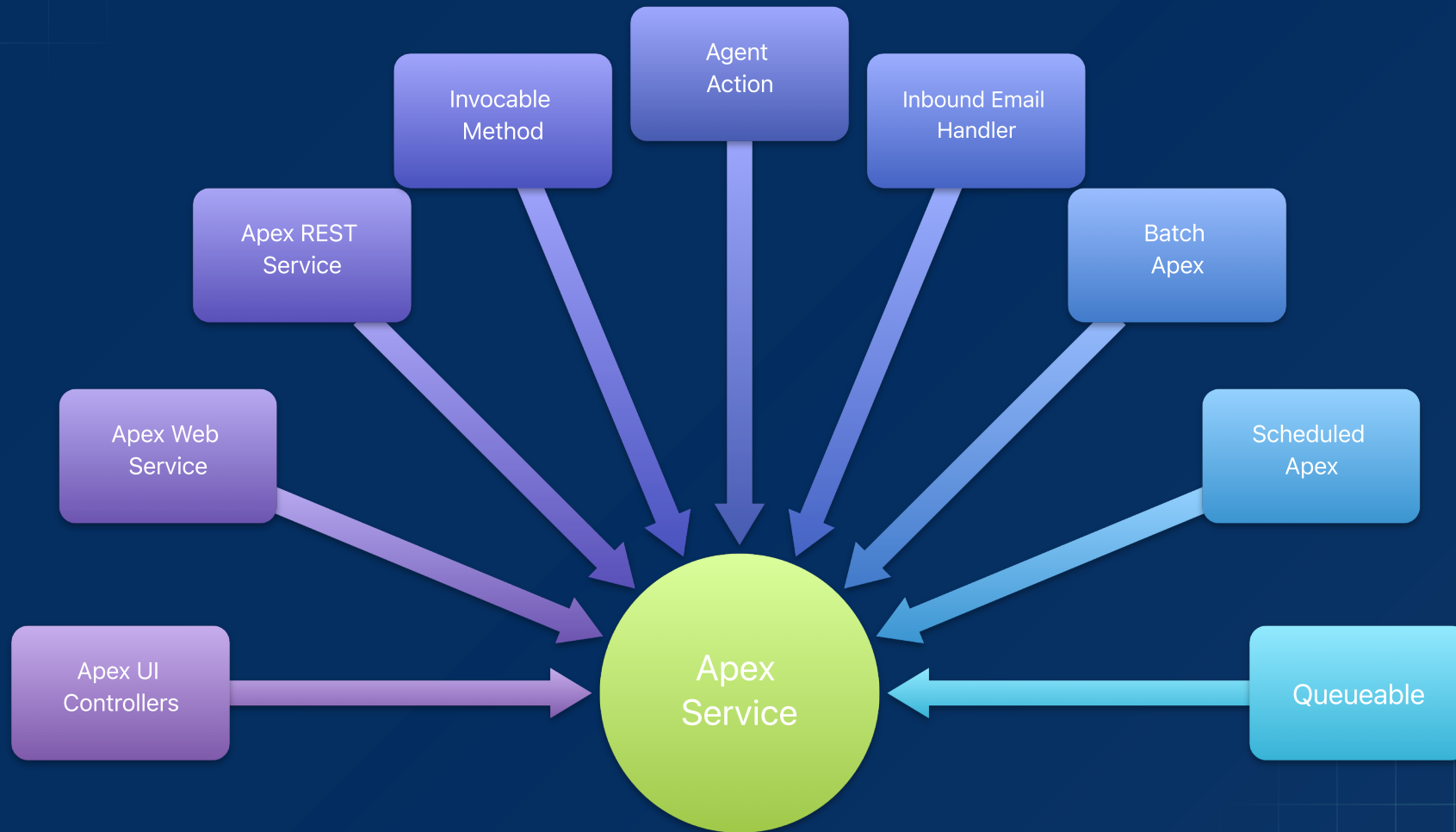
Session abstract

Apex Enterprise Patterns (fflib) - Color Chart



-  **Client**
-  **Service**
-  **Domain**
-  **Trigger Handler**
-  **Selector**

Services have many Consumers



```
public virtual void [methodName]([parameters]) {
```

1. **Service** — [Coordinate the operation]
2. **Selector** — [Load the records]
3. **Domain** — [Apply the object-specific behavior]
4. **Service** — [Complete the transaction]

```
}
```

```
1 // [Paste the focused Apex example here.]
2 //
3 // [Introduce the records or inputs.]
4 // [Show the behavior this section explains.]
5 // [Highlight the important decision.]
6
```

Warehouse App Domain vs Trigger Code

[Diagram / Illustration]

```
1 // [Paste the focused Apex example here.]
2 //
3 // [Introduce the records or inputs.]
4 // [Show the behavior this section explains.]
5 // [Highlight the important decision.]
6
```

Warehouse App Domain vs Trigger Code

- **[Scenario one]**
 - [Question for the audience]
- **[Scenario two]**
 - [Question for the audience]
- **[Discussion takeaway]**
 - [Reveal or summarize the reasoning]

Warehouse App Objects and Behaviors

- **[Principle one]**
 -  [Example that follows the principle]
 -  [Example to avoid]
- **[Principle two]**
 -  [Example that follows the principle]
 -  [Example to avoid]
- **[Principle three]** – [Short explanation]